Design and Development of Li-Fi Transceiver System Using Computer and Mobile Phones

Palwasha Binte Inam,Manahil Faisal,Sara bint Bilal,Rafay Saeed
Department of Electrical Engineering
Ghulam Ishaq Khan Institute of Engineering Sciences and Technology
Email: u2022491@giki.edu.pk ,u2022675@giki.edu.pk,u2022675@giki.edu.pk,u2022497@giki.edu.pk

***Abstract*—This report outlines the design and development of a Li-Fi transceiver system utilizing computer and mobile phone resources. Li-Fi, or Light-Fidelity, is an emerging communication technology that leverages visible light for data transmission. The objective of this project is to implement a prototype system capable of encoding, transmitting, and decoding messages via light signals, employing available tools such as ambient light sensors and appropriate data acquisition techniques.**

I. Introduction

Wireless communication has become an integral part of modern life. While technologies like Bluetooth and Wi-Fi dominate, Light-Fidelity (Li-Fi) presents a novel approach to data transmission by utilizing the visible light spectrum. This report focuses on the implementation of a Li-Fi transceiver system using accessible resources such as personal computers, laptops, and smartphones. The design involves encoding text into binary, transmitting it via modulated light, and decoding the received signal back into text.

II. Problem Statement

The project entails designing and implementing a Li-Fi transceiver system that encodes a text message, converts it to binary, and transmits it through modulated light signals generated by a PC or laptop screen. Smartphones equipped with ambient light sensors are used to receive and decode the transmitted signals. Key challenges include the accurate conversion of lux readings to binary data, the design of an efficient transducer, and the minimization of bit error rates.

III. System Design

A. Literature Review

Relevant techniques in data encoding, modulation, and decoding were reviewed to ensure a robust system design. Li-Fi principles, ADC (Analog-to-Digital Conversion), and DAC (Digital-to-Analog Conversion) methods were studied in detail.

B. Algorithm Design

An algorithm was developed to encode text into binary data, modulate the light signals, and decode the received lux data. The main steps include:

- Text-to-binary conversion using an ADC.
- Binary modulation through the PC/laptop screen.
- Lux data acquisition using a smartphone ambient light sensor.
- Lux-to-binary decoding and binary-to-text conversion using a DAC.

IV. Implementation

A. Stage 1: Hardcoding the Initials into the Li-Fi System

The process of hardcoding initials into the Li-Fi transceiver system involves multiple steps that translate text data into light signals and back into text using digital communication principles. This section outlines each of these steps with a detailed explanation of the code implementation.

B. Assigning ASCII Codes to Initials

The first step involves converting each character into its corresponding ASCII code. Each character (initial) is assigned a unique numerical value according to the ASCII standard. For instance, the letter 'A' corresponds to ASCII code 65, while 'B' corresponds to ASCII code 66. This conversion is achieved using the `ord()` function in Python, which retrieves the ASCII value for each character.

```
ascii_value = ord(char)  % Get ASCII value
```

For example, the letter 'A' is converted to the ASCII value 65.

C. Converting ASCII Codes to Binary

After obtaining the ASCII code for each character, the next step is to convert these numerical values into their binary equivalents. This is done by using the `format()` function in Python, which converts the ASCII value into an 8-bit binary representation.

```
binary_value = format(ascii_value, '08b')
```

For example, the ASCII value of 'A' (65) is converted to the binary representation `01000001`. This binary value will later be used to generate light signals in the Li-Fi system.

D. Synthesizing the Binary Data

Once each character is converted into its binary representation, the binary values are stored in a list and joined into a single string for transmission. The following Python code

synthesizes the binary data for all characters in the input message:

```
binary_data.append(binary_value)
```

For example, if the input message is "pmrs", the binary sequence for this message would be a concatenation of the individual binary values of each character.

E. Flashing the Binary Data via Light

The next step involves encoding the binary data into light signals. This is achieved by simulating light pulses using a graphical user interface (GUI) with Python's Tkinter library. For each bit in the binary data:

- If the bit is 1, the light is on (represented by a white screen).
- If the bit is 0, the light is off (represented by a gray screen).

This is done in the blink_screen function, which controls the screen's color based on the binary data:

```
if bit == '1':
canvas.configure(bg="white")  % Light on
elif bit == '0':
canvas.configure(bg="gray")  % Light off
```

The screen blinks for each bit with a specified duration for the "on" and "off" states.

F. Receiving the Light Signal

The light pulses are received by Firefox app, which records variations in light intensity corresponding to the transmitted binary data.Firefox app is used to detect light intensity and convert it back into a binary sequence. Although not explicitly covered in the code, this is the part where the smartphone captures the light signal and processes it for further decoding.

G. Processing the Received Signal

The received light signal, captured by the photoreceptor, is then processed to decode the binary values. An algorithm on the smartphone or other receiving device converts the binary sequence back into ASCII codes and then into the original initials or message. This is the reverse process of encoding, where the binary data is translated back into text.

H. Modified Data Representation

In the code, an additional modification is applied to the binary data by appending the character '2' to each binary bit. This is done to create a new binary sequence:

```
modified_message.append(binary_message[i])
```

This modification does not affect the underlying binary transmission but could be used for additional signaling or error detection purposes. The modified message is then transmitted using the same light pulse encoding process.

I. Stage 2: Processing the Recorded Signal (ADC)

In Stage 2, we process the recorded lux values, which were captured during the transmission of light signals. The goal is to convert these analog lux values back into digital data by identifying the binary 1s and 0s in the light signal using an Analog-to-Digital Converter (ADC) process. This stage involves importing the recorded lux data from a CSV file, generating a graph, identifying peaks corresponding to binary values, and filtering out improper lux values.

J. Importing the Recorded Signal

The first step in Stage 2 is to import the recorded lux values from the CSV file into the laptop. These lux readings were captured by the smartphone's ambient light sensor, which recorded the variations in light intensity as the binary data was transmitted via the Li-Fi system.

In Python, this is done using the pandas library, which allows us to read and process the CSV file containing the lux data. The data is then stored in a variable for further processing:

```
import pandas as pd

# Load the lux data from the CSV file
lux_data = pd.read_csv('lux_data.csv')
```

This step provides us with the raw light intensity measurements that were captured during the signal transmission.

K. Generating the Graph

After importing the lux data, we use Python's matplotlib library to generate a graph that visualizes the lux readings over time. This graph helps in identifying the peaks (high light intensity) and valleys (low light intensity) that represent the binary 1s and 0s, respectively.

```
import matplotlib.pyplot as plt

# Plotting the lux values
plt.plot(lux_data['lux'])
plt.title("Lux vs Time")
plt.xlabel("Time (data points)")
plt.ylabel("Lux (light intensity)")
plt.show()
```

The x-axis represents the time or data points, while the y-axis represents the lux values (light intensity). Peaks in the graph correspond to high light intensity (binary 1), and valleys correspond to low light intensity (binary 0).

L. Identifying Max and Min Peaks

In the graph, the peaks and valleys are analyzed to determine where the binary 1s and 0s occur. The argrelextrema function from the scipy library can be used to find the local maxima (peaks) and minima (valleys) in the lux data.

```
import numpy as np
```

```
from scipy.signal import argrelextrema
max_peaks = argrelextrema
min_peaks = argrelextrema(lux_data)
```

The maxima represent the light intensity corresponding to a binary 1, while the minima represent the light intensity corresponding to a binary 0. These points are critical in recovering the original message from the lux data.

M. Filtering Out Improper Lux Values

After identifying the peaks and valleys, we must filter out any improper lux values. These improper values may result from noise, interference, or other distortions in the signal. To ensure the integrity of the transmitted data, we remove values that do not correspond to expected binary thresholds.

We can filter the lux data by applying a threshold to remove values that are too low (representing noise) or too high (representing erroneous peaks). The following code shows an example of how to filter out improper lux values based on a predefined threshold:

```
threshold_high = 10
threshold_low = 7


filtered_lux = lux_data[]
```

This filtering process ensures that only valid lux values, which represent binary 1s and 0s, are retained for further processing.

N. Simulating the ADC Process

By identifying the peaks and valleys in the lux data, filtering out improper values, and applying a threshold, we simulate the process of an Analog-to-Digital Converter (ADC). The ADC converts the analog light signal (lux data) into discrete digital data (binary 1s and 0s) that can be used to reconstruct the transmitted message.

The binary data is extracted by analyzing the lux values and comparing them against the threshold values. Once the proper peaks and valleys are identified, the signal is decoded back into binary form.

O. Conclusion

In Stage 2, the recorded lux signal is processed to extract the binary data by analyzing the graph of lux values, identifying the max and min peaks corresponding to 1s and 0s, and filtering out improper lux values to remove noise. This simulates the ADC process, which is crucial for converting the analog light signal into discrete digital data. The clean binary data can then be decoded back into the original message or initials, completing the Li-Fi communication cycle.

P. Stage:3 Converting Binary Data to String

In Stage 3, the goal is to convert the cleaned binary data received from the light signal transmission, which has been processed and filtered in Stage 2, back into the original message or string. This stage focuses on interpreting the binary values, representing the light signal, and converting them into human-readable characters.

Q. Generating the Cleaned Data Graph

After the lux values have been filtered and cleaned in Stage 2, a new graph is generated to represent the cleaned binary data. This graph highlights the peaks (which correspond to binary 1s) and valleys (which correspond to binary 0s) in the received light signal. The cleaned graph visually represents the final sequence of binary digits that were transmitted and received.

The following code demonstrates how the cleaned data is plotted with local maxima (representing binary 1s) highlighted:

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.signal import find_peaks

# Read CSV file
def read_csv_file(file_path):
try:
data = pd.read_csv(file_path, header=0)
return data
except FileNotFoundError:
print("File not found")
return None
except pd.errors.EmptyDataError:
print("No data in file.")
return None
```

The code reads the CSV file, extracting the data, which contains the lux values captured during transmission.

R. Receiving the Binary Data

The system receives the cleaned binary data, which represents the transmitted light signal. At this point, the data has already been converted into a digital form, corresponding to the variations in light intensity that were transmitted via the Li-Fi system. This data is now in the form of binary digits (1s and 0s).

The following code finds the local maxima (peaks) in the lux data, which represent binary 1s:

```
# Find local maxima
def find_local_maxima(data):
y_values = data.iloc[:, 1].values
peaks, _ = find_peaks(y_values)
return peaks
```

The code identifies the indices of the local maxima in the lux data, where peaks are found in the light intensity measurements.

S. Converting Binary Data to String

Once the binary data is received and cleaned, the next step is to convert it back into a human-readable string. Each 8-bit binary value represents one ASCII character. For example, a sequence of eight 1s and 0s like `01000001` corresponds to the letter 'A' in ASCII.

The system processes the binary data by grouping it into 8-bit chunks and converting each chunk into its corresponding ASCII character. This is done using Python's `int()` function to convert the binary string into an integer, and the `chr()` function to convert that integer into the character.

```
def binary_to_string(binary_data):

characters = []
for i in range(0, len(binary_data), 8):
byte = binary_data[i:i+8]
char = chr(int(byte, 2))
characters.append(char)

return ''.join(characters)
```

The `binary_to_string()` function splits the binary data into 8-bit chunks, converts each chunk into a character, and concatenates the resulting characters to form the decoded message.

T. Reconstructing the Original Message

After converting each 8-bit binary chunk into its corresponding ASCII character, the system reconstructs the original string by concatenating the decoded characters. This message is identical to the one that was initially transmitted via the Li-Fi system.

The following code converts the cleaned binary data into a readable string:

```
# Main function
def main():
file_path = input("Enter CSV file path: ")
data = read_csv_file(file_path)

if data is not None:
maxima_indices = find_local_maxima(data)
{data.iloc[maxima_indices].values}")

filter_maxima(up_thresh=7, low_thresh=2.5)
```

The main function calls all the necessary steps, from reading the CSV file to finding the local maxima, filtering the maxima, converting the filtered binary data into a string, and plotting the cleaned data graph.

U. Conclusion

In Stage 3, the cleaned binary data, which was received from the Li-Fi transmission, is processed to convert it back into a human-readable string. The system achieves this by identifying the local maxima (which represent binary 1s), filtering the data, and converting the cleaned binary data into its ASCII representation. This completes the decoding process and reconstructs the original message or initials that were transmitted via the Li-Fi system.

V. Result of Execution of the Code

This process demonstrates how initials are encoded into light signals and decoded back into text using the principles of digital communication and Li-Fi technology. The Python code simulates the key steps involved in this process, including converting text to binary, transmitting binary data as light pulses, and receiving and decoding the light signals to recover the original message.

VI. Algorithim Flow

```
# Main script
if __name__ == "__main__":
    message = "pmrs"
    binary_message = dac_convert_to_binary(message)  # Get the binary sequence usir
    modified_message = []
    modified_message.append('2222222222222222')
    for i in range(len(binary_message)):
        modified_message.append(binary_message[i] + '2')
    modified_message.append('2222222222222222')

    print(f"Binary representation of '{message}': {binary_message}")

    modified_message = ''.join(modified_message)
    print(modified_message)
```

Fig. 1. Stage 1

```
# Highlight maxima
plt.scatter(x_values.iloc[maxima_indices], y_values.iloc[maxima_indices],
            color='red', label="Local Maxima", zorder=5)

plt.xlabel("Time (s)")
plt.ylabel("Illuminance (lx)")
plt.title("Illuminance over Time with Local Maxima")
plt.grid(True)
plt.legend()
plt.show()
```

Fig. 2. Stage 2

```
# Filter maxima
def filter_maxima(data, maxima_indices, upper_threshold, lower_threshold):
    maxima_values = data.iloc[maxima_indices].values
    filtered_maxima = [1 if value[1] > upper_threshold else 0 if value[1] > lower_threshold else 2 for value in
    return filtered_maxima

def binary_to_string(binary_data):
    # Ensure binary data length is a multiple of 8
    if len(binary_data) % 8 != 0:
        raise ValueError("Binary data length must be a multiple of 8.")

    characters = []
    for i in range(0, len(binary_data), 8):
        byte = binary_data[i:i+8]  # Extract 8 bits
        char = chr(int(byte, 2))  # Convert binary to character
        characters.append(char)

    return ''.join(characters)
```

Fig. 3. Stage 3

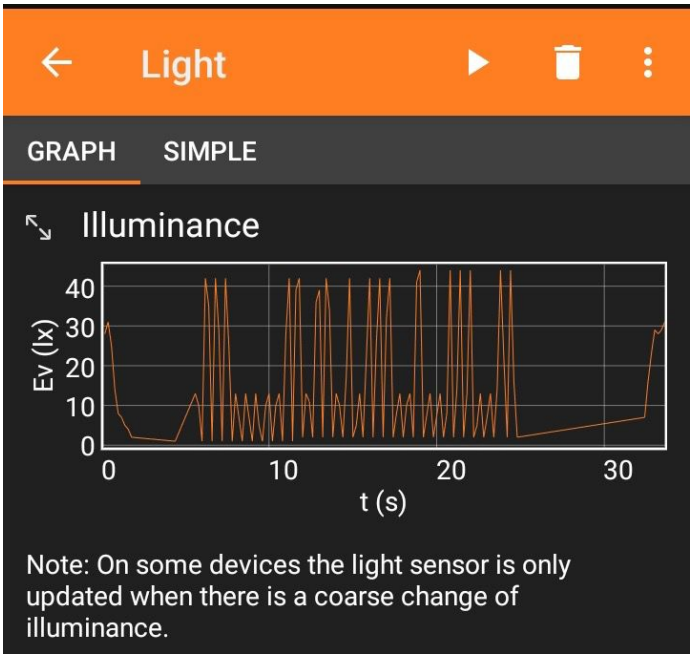

Fig. 4. Captured Data

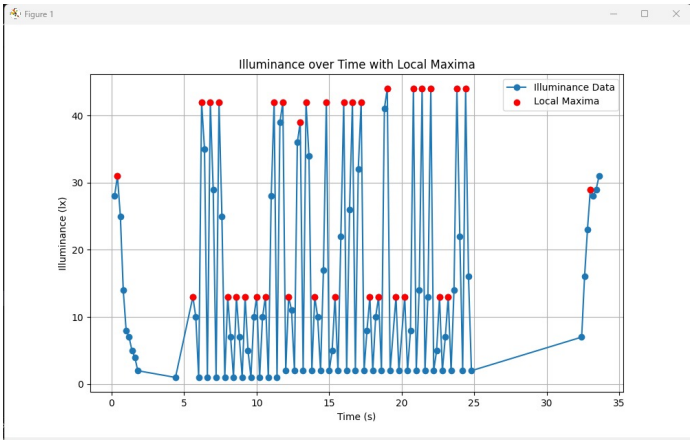

Fig. 5. Data to be cleaned

VII. Troubleshooting

Initially, we encountered issues with data transmission using a visual method that involved blinking the screen in black and white. Without a separator between bits, the result was a continuous wave where individual bits of data were indistinguishable, leading to garbage values.

To resolve this, we introduced a separator bit (value 2) represented by black. Additionally, we assigned specific RGB values to distinguish the other bits.

This approach added structure to the transmission, making it possible to distinguish individual bits. As a result, data transmission and reception became accurate and reliable.

VIII. Expected Outcomes

The following outcomes are anticipated:

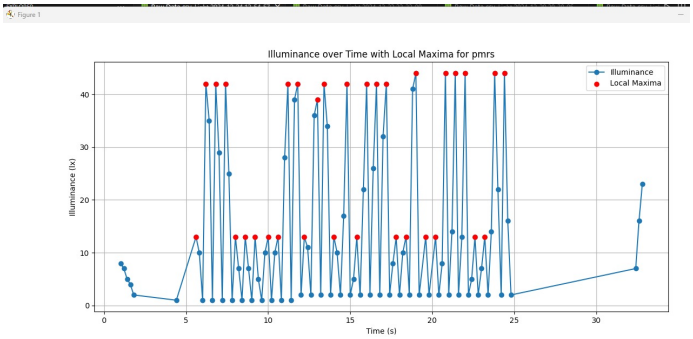

Fig. 6. Output

- A working prototype of a Li-Fi transceiver system.
- Demonstration of Li-Fi principles, including data encoding, modulation, and decoding.
- Analysis of system performance under varying ambient light conditions.

IX. Conclusion

The project provides an opportunity to explore the principles and applications of Li-Fi technology using readily available tools. The implementation of the proposed system highlights the potential of Li-Fi as a viable alternative to traditional wireless communication methods.

X. References

1) H. Haas, "LiFi: Conceptions, Misconceptions, and Opportunities," *IEEE Communications Magazine*, vol. 55, no. 5, pp. 177-183, May 2017.
2) T. Komine and M. Nakagawa, "Fundamental Analysis for Visible-Light Communication System using LED Lights," *IEEE Transactions on Consumer Electronics*, vol. 50, no. 1, pp. 100-107, Feb. 2004.
3) S. Rajagopal, R. D. Roberts, and S. K. Lim, "IEEE 802.15.7 Visible Light Communication: Modulation Schemes and Dimming Support," *IEEE Communications Magazine*, vol. 50, no. 3, pp. 72-82, March 2012.

XI. ACKNOWLEDGEMENT

The authors would like to thank Dr Nisar Ahmed and Sir Asad Malik for their contiunous guidance and support.